Multiplayer Game Platform — Initial Project Plan

Developed by Bogo Solutions

Overview

Three one-week sprints covering the full implementation of the platform as designed in the architecture document. Each sprint is organized by sub-team: **Platform Core**, **Client/UI**, **Rules & Validation**, and **Quality, Testing & Integration**.

Sprint 1 builds the foundation. Sprint 2 implements the core platform features. Sprint 3 integrates everything, closes out stats and notifications, and prepares the demo.

Table of Contents

1. Sprint 1 — Foundation & Core Abstractions
2. Sprint 2 — Core Platform Features
3. Sprint 3 — Integration, Notifications & Demo
4. Dependency Summary
5. Product Backlog

Sprint 1 — Foundation & Core Abstractions (Week 1)

Goal: Establish the project skeleton, all shared interfaces and data models, authentication flows, and stub boundaries so every sub-team can work in parallel from Week 2 onward.

Platform Core

Task
Project structure setup (`ui` , `platform` , `game` , `model` packages)
Define `Game` interface
Define `RuleValidator` interface
Define `PlayerAction` abstraction
Define `GameStateUpdate` event class
Define `GameStatus` enum and `GameState` class

Task
Implement `EventBus`
Stub: `CredentialsDatabase`
Stub: `Database`
Stub: `GameServer` and `Networking`

Client/UI

Task
`WelcomePage`
`AuthenticationScreen`
`LoginPage`
`State` enum

Rules & Validation

Task
`Player` domain model
`GeneralStats` class
`Rank` enum
`PlayerStats` class
`Settings` class

Quality, Testing & Integration

Task
Issue Board Setup
README
Branching convention
Test infrastructure setup

Task
Unit tests: `Player`
Unit tests: `GeneralStats`
Unit tests: `CredentialsDatabase` stub

Sprint 2 — Core Platform Features (Week 2)

Goal: Implement the full matchmaking-to-game-session pipeline, the move handling pipeline, both game implementations, and the primary UI screens. By end of this sprint the lobby → game start → turn progression flow should be runnable end-to-end.

Platform Core

Task
`SessionManager`
`PlatformCore` — auth delegation (login & register)
`PlatformCore` — dispatch & notification routing
`PlatformCore` — queue proxy (`joinQueue`)
`PlatformCore` — lobby proxy (`joinLobby`)
`PlatformCore` — move proxy (`submitMove`)
`MatchMaking` service
`Lobby` service (`createLobby` , `getAvailableLobbies` , `addPlayerToLobby`)
`GameSessionManager`
`Leaderboard` service (skeleton)

Client/UI

Task
`HomePage`
`MainGameDashboard`
`ProfileDashboard`

Task
`PlayGame`
`GameGUI`
`ChatboxGUI`

Rules & Validation

Task
`TicTacToeGame` (implements `Game`)
`TicTacToePlayerAction`
`TicTacToeRuleValidator` (implements `RuleValidator`)
`TicTacToeGameState`
`ConnectFourGame` (implements `Game`)
`ConnectFourPlayerAction`
`ConnectFourRuleValidator` (implements `RuleValidator`)
`ConnectFourGameState`

Quality, Testing & Integration

Task
Merge Sprint 1 branches
End-to-end test
Update architecture doc
Unit tests: `TicTacToeRuleValidator`
Unit tests: `ConnectFourRuleValidator`
Unit tests: `GameSessionManager`
Unit tests: `MatchMaking`
Integration test: Login flow
Integration test: Register flow

Sprint 3 — Integration, Notifications & Demo (Week 3)

Goal: Complete the notification system and stats pipeline, resolve all integration issues, implement the full game-over flow, wire up the leaderboard, and prepare the demo deliverable.

Platform Core

Task
`Chat` service (`sendMessage` , `filterMessage`)
`PlatformCore` — notification routing (complete)
`StatsManager`
`Leaderboard` (complete)
Game-over pipeline (complete)

Client/UI

Task
`Settings` UI
Win/loss/tie result screen
Turn alert display
Player statistics view
Leaderboard view
UI polish & navigation wiring

Rules & Validation

Task
Win detection — Tic-Tac-Toe
Win detection — Connect Four
Valid actions delivery
Validation failure reason propagation

Quality, Testing & Integration

Task
Merge Sprint 2 & 3 branches
End-to-end demo run
Final README
Architecture doc update
Stub documentation
Integration test: full game flow
Integration test: turn notification
Integration test: game-over stats
Unit tests: `StatsManager`
Unit tests: win detection
Demo script verification

Dependency Summary

The table below shows which Sprint 2 and 3 tasks have hard dependencies on Sprint 1 deliverables.

Sprint 2 / 3 Task	Depends On (Sprint 1)
`PlatformCore` auth	`CredentialsDatabase` stub, `SessionManager` skeleton, `Player` model
`GameSessionManager`	`Game` interface, `EventBus`, `GameStateUpdate`
`TicTacToeGame` / `ConnectFourGame`	`Game` interface, `RuleValidator` interface, `PlayerAction` abstraction
`MatchMaking`	`Player` model, `Database` stub
`Lobby`	`Player` model, `GameStatus` enum
`Chat`	`Lobby` service
`StatsManager`	`GeneralStats`, `Rank`, `Database` stub
`PlatformCore` notification routing	`GameStateUpdate`, `EventBus`, `GameStatus` enum

Sprint 2 / 3 Task	Depends On (Sprint 1)
All UI screens	`PlatformCore` API surface, `Player` model, `GameStateUpdate`

Product Backlog

All known work items for the platform, ordered by priority. Items are marked with their assigned sprint and sub-team. Backlog items not yet assigned to a sprint are candidates for scope adjustment or future iterations.

Priority levels: `High` — required for a working platform demo. `Medium` — significantly improves completeness or quality. `Low` — nice-to-have; deferred if time is short.

Authentication & Session Management

ID	Item	Priority	Sprint	Sub-Team
AUTH-01	`CredentialsDatabase` stub with hardcoded test accounts and bcrypt password storage	High	1	Platform Core
AUTH-02	`SessionManager` : create, store, and clear session tokens	High	2	Platform Core
AUTH-03	`PlatformCore` login flow: verify credentials → create session → return token	High	2	Platform Core
AUTH-04	`PlatformCore` register flow: check username availability → create account → create session	High	2	Platform Core
AUTH-05	Login UI: `WelcomePage` , `AuthenticationScreen` , `LoginPage` with success/failure handling	High	1	Client/UI
AUTH-06	`Settings` UI: change username, email, password, and logout	Medium	3	Client/UI
AUTH-07	`Settings` class: `deleteAccount` , account field updates, references `CredentialsDatabase`	Medium	1	Rules & Validation

Player Profiles & Statistics

ID	Item	Priority	Sprint	Sub-Team
PROF-01	`Player` domain model: all fields and methods	High	1	Rules & Validation

ID	Item	Priority	Sprint	Sub-Team
PROF-02	`GeneralStats` class: win/loss/tie tracking, MMR, and rank update logic	High	1	Rules & Validation
PROF-03	`Rank` enum: seven tiers from `BRONZE` to `GRANDMASTER`	High	1	Rules & Validation
PROF-04	`PlayerStats` class: per-game stat getters delegating to `GeneralStats`	Medium	1	Rules & Validation
PROF-05	`StatsManager` : parse game result, update `GeneralStats` , push to `Database` stub	High	3	Platform Core
PROF-06	`ProfileDashboard` UI: display profile view and per-game stats	Medium	2	Client/UI
PROF-07	Player statistics view: display stats separated by game type	Medium	3	Client/UI

Matchmaking & Lobby

ID	Item	Priority	Sprint	Sub-Team
MM-01	`MatchMaking` service: enqueue, dequeue, match when minimum players reached	High	2	Platform Core
MM-02	`Lobby` service: `createLobby` , `getAvailableLobbies` , `addPlayerToLobby`	High	2	Platform Core
MM-03	`PlatformCore` queue proxy: `joinQueue()` with player eligibility check	High	2	Platform Core
MM-04	`MainGameDashboard` UI: game selection, queue entry, leaderboard entry point	High	2	Client/UI
MM-05	Player eligibility check before queue entry (ban status, score constraints)	Medium	2	Platform Core

Game Session & Turn Handling

ID	Item	Priority	Sprint	Sub-Team
SESS-01	`Game` interface: six core methods	High	1	Platform Core
SESS-02	`RuleValidator` interface: `validate()` with failure reason, `getValidActions()`	High	1	Platform Core

ID	Item	Priority	Sprint	Sub-Team
SESS-03	`PlayerAction` abstraction: player ID, action type, action payload	High	1	Platform Core
SESS-04	`GameStateUpdate` event class: all fields	High	1	Platform Core
SESS-05	`GameStatus` enum and `GameState` class	High	1	Platform Core
SESS-06	`EventBus` : `publish()` and `subscribe()`	High	1	Platform Core
SESS-07	`GameSessionManager` : `startSession` , `submitMove` pipeline, `endSession`	High	2	Platform Core
SESS-08	`GameServer` stub: `createSession()` returning generated session ID	High	1	Platform Core
SESS-09	Game-over pipeline: `isGameOver()` → `StatsManager` → publish `ENDED` event → return to dashboard	High	3	Platform Core

Notifications

ID	Item	Priority	Sprint	Sub-Team
NOTIF-01	`PlatformCore` dispatch: route `GameStateUpdate` to correct player based on current screen	High	2	Platform Core
NOTIF-02	`Chat` service: `sendMessage()` , optional `filterMessage()` hook	Medium	3	Platform Core
NOTIF-03	Turn alert display in `MainGameDashboard` : banner/badge when player's turn is ready	High	3	Client/UI

Game Implementations

ID	Item	Priority	Sprint	Sub-Team
GAME-01	`TicTacToeGame` : full `Game` interface implementation	High	2	Rules & Validation
GAME-02	`TicTacToePlayerAction` : target cell position payload	High	2	Rules & Validation
GAME-03	`TicTacToeRuleValidator` : empty cell check, turn check, failure reason, valid actions	High	2	Rules & Validation

ID	Item	Priority	Sprint	Sub-Team
GAME-04	`TicTacToeGameState` : 3×3 board, current player, move history	High	2	Rules & Validation
GAME-05	Tic-Tac-Toe win/draw detection: row, column, diagonal, full-board draw	High	3	Rules & Validation
GAME-06	`ConnectFourGame` : full `Game` interface implementation	High	2	Rules & Validation
GAME-07	`ConnectFourPlayerAction` : target column payload	High	2	Rules & Validation
GAME-08	`ConnectFourRuleValidator` : column full check, turn check, failure reason, valid actions	High	2	Rules & Validation
GAME-09	`ConnectFourGameState` : 6×7 board, current player, move history	High	2	Rules & Validation
GAME-10	Connect Four win detection: horizontal, vertical, diagonal four-in-a-row	High	3	Rules & Validation
GAME-11	Valid actions included in `GameStateUpdate.payload` for client-side move disabling	Medium	3	Rules & Validation
GAME-12	Validation failure reason propagated through `GameSessionManager` to UI error display	Medium	3	Rules & Validation

UI & Navigation

ID	Item	Priority	Sprint	Sub-Team
UI-01	`WelcomePage` : entry screen with navigation to login/register	High	1	Client/UI
UI-02	`AuthenticationScreen` : login and registration forms	High	1	Client/UI
UI-03	`LoginPage` : credential submission, success/failure callbacks, `State` enum	High	1	Client/UI
UI-04	`HomePage` : own records, other player records, settings, game selection	Medium	2	Client/UI
UI-05	`PlayGame` : game board display, move submission, winner display	High	2	Client/UI

ID	Item	Priority	Sprint	Sub-Team
UI-06	`GameGUI` : window setup, board render, UI update	High	2	Client/UI
UI-07	`ChatboxGUI` : message display and chatbox	Low	2	Client/UI
UI-08	Win/loss/tie result screen with updated rank display	High	3	Client/UI
UI-09	Leaderboard view in `MainGameDashboard`	Medium	3	Client/UI
UI-10	Full navigation wiring: no dead ends across all screen transitions	High	3	Client/UI

Leaderboard

ID	Item	Priority	Sprint	Sub-Team
LB-01	`Leaderboard` service skeleton: `logGameResult` , `updatePlayer` , `getTopPlayersByRank`	Medium	2	Platform Core
LB-02	`Leaderboard` wired to `StatsManager` and `Database` stub	Medium	3	Platform Core
LB-03	`Database` stub: `storeData()` , `retrieveData()` in-memory implementation	High	1	Platform Core

Quality, Testing & Integration

ID	Item	Priority	Sprint	Sub-Team
QA-01	GitLab repo setup: branch protection, TA/instructor access, branching conventions	High	1	QT&I
QA-02	README: build and run instructions, project overview, sub-team responsibilities	High	1	QT&I
QA-03	JUnit test infrastructure: package structure mirroring `src`	High	1	QT&I
QA-04	Unit tests: `Player` , `GeneralStats` , `CredentialsDatabase` stub	High	1	QT&I

ID	Item	Priority	Sprint	Sub-Team
QA-05	Merge Sprint 1 branches into `main`	High	2	QT&I
QA-06	Smoke test: launch → login → dashboard	High	2	QT&I
QA-07	Architecture doc update after Sprint 1 implementation	Medium	2	QT&I
QA-08	Unit tests: `TicTacToeRuleValidator`, `ConnectFourRuleValidator`	High	2	QT&I
QA-09	Unit tests: `GameSessionManager`, `MatchMaking`	High	2	QT&I
QA-10	Integration tests: login flow, register flow	High	2	QT&I
QA-11	Final merge of all Sprint 2 & 3 branches into `main`	High	3	QT&I
QA-12	Integration test: full game flow end-to-end	High	3	QT&I
QA-13	Integration test: turn notification while player is in lobby	High	3	QT&I
QA-14	Integration test: game-over stats and rank update	High	3	QT&I
QA-15	Unit tests: `StatsManager`, win detection for both games	High	3	QT&I
QA-16	Demo script verification: all required flows, no crashes	High	3	QT&I
QA-17	Final README, stub documentation, architecture doc final revision	Medium	3	QT&I

Plan version: aligned with architecture document v1.0.0. Adjust task assignments to sub-team members at the start of each sprint.